MongoDB

Ecosistema MongoDB

Giorno 2 · pomeriggio

Marco De Giovanni · 21 ottobre 2026

PO PROGRAMMA DEL POMERIGGIO

Due cicli: teoria e laboratorio

Sequenza

Teoria 3	Modellazione, transazioni
Laboratorio 3	Modellazione
Teoria 4	Indici, `explain()`, aggregation e view, confronto con PostgreSQL
Laboratorio 4	Indici e aggregation
Q&A e chiusura	

RIPARTENZA

All'inizio del pomeriggio il container è acceso e Compass connesso (stesso controllo della mattina).

AMBIENTE

MongoDB **8.0.32**, dataset ShopIT. Il laboratorio si svolge con gli esercizi delle slide **L3** e **L4**.

01

CICLO 3 · TEORIA

Modellazione, transazioni

Il cambio di paradigma e i pattern di accesso

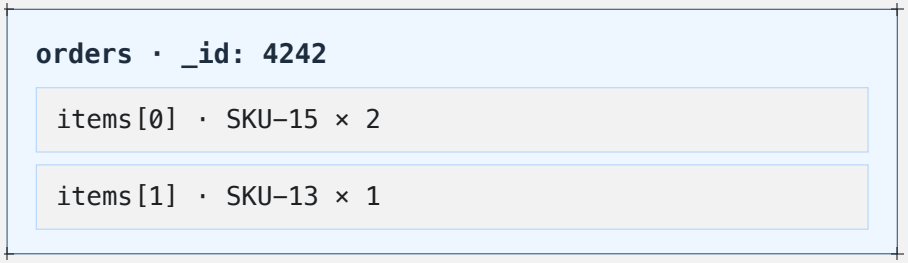
	RELAZIONALE (GIORNO 1)	DOCUMENTALE
Si parte da	le **entità** e le loro dipendenze	le **operazioni** dell'applicazione
Obiettivo	nessuna ridondanza (3NF)	una lettura = un documento
Le relazioni	si risolvono in lettura (`JOIN`)	si risolvono in scrittura (embedding, copie)
Se cambiano le query	un indice o una vista	può servire rimodellare

Si progetta con una tabella dei carichi: per ogni operazione, *cosa* si legge o si scrive insieme, *quanto spesso*, *quanti* elementi (e se crescono), *quanto aggiornato* deve essere il dato letto.

Embedding vs referencing: vantaggi e limiti

	EMBEDDING (ANNIDARE)	REFERENCING (RIFERIRE)
Vantaggi	una lettura, **atomicità gratuita**, località dei dati, forma dell'oggetto	un dato in un posto solo, documenti piccoli, entità interrogabili da sole, cardinalità illimitata
Limiti	il documento **cresce** (16 MB), si legge sempre tutto, dati annidati poco interrogabili da soli, duplicazione se annidati in più documenti	più letture o `$lookup`, **nessuna integrità referenziale**, modifiche su più documenti atomiche solo con transazione

EMBEDDING



1 lettura, 1 scrittura atomica

REFERENCING



più letture o `$lookup`

Criteri di scelta e domande guida

DOMANDA	EMBEDDING	REFERENCING
I dati si leggono insieme?	sì	no, o raramente
Cardinalità del lato "molti"	limitata (decine)	grande o illimitata
Il dato annidato ha vita propria?	no	sì
Il dato annidato cambia spesso?	no	sì
Serve atomicità tra i due?	sì (gratuita)	solo con transazione

DOMANDE GUIDA

01
Quali operazioni e con che frequenza?

02
Cosa si legge insieme?

03
Qual è la cardinalità massima, e cresce?

04
Cosa ha vita propria o cambia spesso?

05
Cosa va modificato atomicamente?

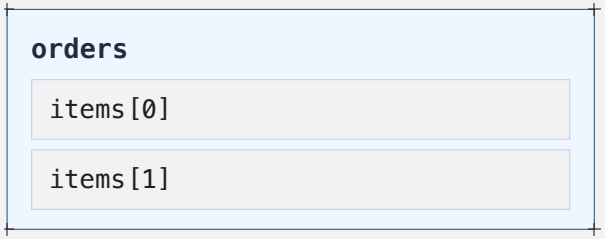
06
Quali dati copiare, con quale semantica (storica o cache)?

1:1 e 1:N: i tre casi

+ **1:1**: quasi sempre **sottodocumento** (`address`, `preferences`). Si separa solo se la parte è grande e letta di rado, ha permessi diversi (in MongoDB i permessi sono per collezione) o un ciclo di vita diverso.

1:pochi

decine



Embedding: i figli dentro il padre

righe di un ordine

1:molti

centinaia, migliaia

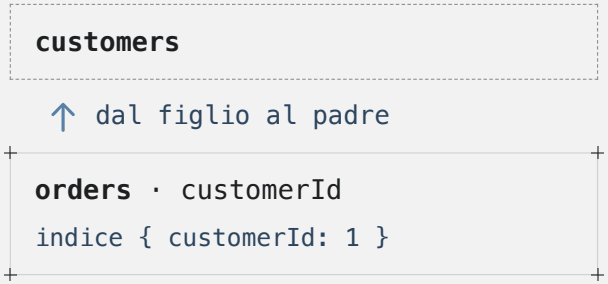


Array di riferimenti nel padre

prodotti di una categoria

1:illimitati

senza limite



Riferimento nel figlio + indice

ordini di un cliente, recensioni

N:N: array di riferimenti, doppio riferimento, collezione ponte

Un lato

array + indice

students
s1 · courseIds: [c1, **c2**]
s2 · courseIds: [**c2**, c3]

↓ courseIds → _id

courses · c1, **c2**, c3 (nessun array)

c2 è di s1 **e** di s2: molti studenti per corso, molti corsi per studente. Il legame è scritto solo negli studenti.

1 scrittura per legame

↓ la scelta di partenza: è il modello dello snippet

Entrambi i lati

due array

students
s1 · courseIds: [c1, c2]

↑↓ legame scritto due volte

courses
c1 · studentIds: [s1, …]

2 scritture per legame → transazione

il lato grande cresce senza limite

Collezione ponte

attributi del legame

students · s1

↑ studentId

enrollments
studentId, courseId
enrolledAt, grade

↓ courseId

courses · c1

Il legame ha dati propri

iscrizione con data e voto

MONGOSH

```
// modello "un lato": l'array courseIds sta solo negli studenti
db.lab_students.createIndex({ courseIds: 1 })
db.lab_students.find({ courseIds: 'c2' }, { name: 1 })        // studenti del corso c2
```

Extended reference: duplicazione controllata e riallineamento

MONGOSH

```
{ _id: 4242, customerId: 46064, customer: { name: 'Customer 46064', country: 'IT' }, … }   // riferimento + copia
```

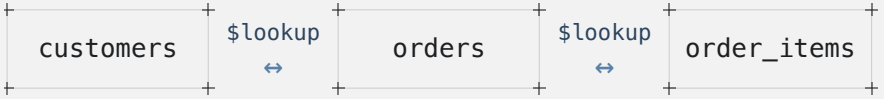
- Si mantiene il **riferimento** e si copiano solo i campi **letti spesso** e **modificati raramente**: si evita il `$lookup` nelle letture frequenti.
- Due semantiche per la copia: **storica** (il nome *al momento dell'ordine*, non si aggiorna mai) o **cache** (va riallineata).
- Riallineamento: **mai**; **subito** (`updateMany`); **in transazione**; **dopo**, in modo asincrono (**change stream** o job).
- Il costo si misura: rinominare il cliente 4242 tocca **10 ordini**, ma senza indice su `customerId` il server ne esamina **1.000.000**.

QUANDO SI RIALLINEA LA COPIA

mai	copia **storica**: il nome al momento dell'ordine
subito	**`updateMany`**, non atomico
in transazione	cliente e copie insieme
dopo	asincrono: **change stream** o job

Anti-pattern: tabelle tradotte e documento illimitato

TABELLE TRADOTTE



✕ ogni lettura dell'ordine richiede `$lookup`

DOCUMENTO ILLIMITATO



orders[0] … orders[n]: ogni ordine riscrive il documento **16 MB**

+ **Il modello relazionale copiato così com'è**: `customers`, `orders`, `order_items` come tre collezioni. Ogni lettura dell'ordine richiede `$lookup`, nessuna atomicità tra ordine e righe senza transazione: né i vantaggi del documentale né le garanzie del relazionale.
+ **Il documento che cresce senza limite**: il cliente con *tutti* i suoi ordini annidati. Ogni ordine riscrive il documento, ogni lettura trasferisce tutto, e prima o poi i 16 MB.

Pattern: subset, computed, bucket, outlier

Subset nel padre solo una parte dei figli

```
products  { _id: 'P-001',
  lastReviews: [ … ultime 5 ] }
reviews   { productId: 'P-001', … }  × 60
```

Costo · recensione scritta due volte

Computed il derivato si calcola in scrittura

```
products  { _id: 'P-001',
  rating: { count: 60, sum: 252 } }
// media in lettura: 252 / 60 = 4,2
```

Costo · ogni recensione fa anche un `$inc`

Bucket tante misure piccole in un documento

```
readings  { sensorId: 'S1', day: '2026-10-21',
  values: [ { t: '00:00', v: 21.4 }, … ] }
// 1 documento al giorno invece di 1440
```

Costo · la singola misura è più scomoda da leggere · dalla 5.0: time series

Outlier il caso estremo va a parte

```
products  { _id: 'P-999', hasOverflow: true }
// 12.000 recensioni: le altre vanno a parte
overflow  { productId: 'P-999', page: 2, items: […]
}
```

Costo · il codice controlla il flag e gestisce il caso speciale

Il modello della giornata

RELAZIONE	CASO	SCELTA
ordine → righe	1:pochi, lette insieme	`items` **embedded**
ordine → attributi variabili	1:1, sparsi	sottodocumento `attrs`
cliente → ordini	1:illimitati	`customerId` **nel figlio** + indice
ordine → nome del cliente	letto spesso, cambia di rado	**extended reference** `customer`
prodotto → recensioni	1:illimitati + lettura frequente delle ultime	collezione `reviews` + **subset** `lastReviews` nel prodotto
prodotto → voto medio	letto spesso	**computed** `rating { count, sum }`

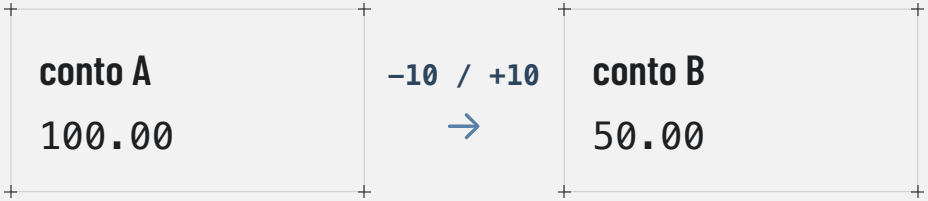
Nel laboratorio 3: partite da un prodotto con le recensioni **annidate** (modello legacy) e lo portate a questo modello.

TX1 CICLO 3 · TRANSAZIONI

L'atomicità del documento e quando serve una transazione

- Un buon modello copre la gran parte dei casi **senza** transazioni: ordine + righe nello stesso documento; controllo e modifica nello stesso update (`{ _id, stock: { $gte: 2 } }`); contatori e valori calcolati con `$inc`.
- La transazione serve quando la modifica tocca **più documenti** che non si possono unire. Il caso classico: il **trasferimento** tra due conti (`−10` su A, `+10` su B).
- Segnale d'allarme: se quasi ogni operazione richiede una transazione, il modello è stato tradotto dal relazionale (M7).

TRASFERIMENTO



Se si interrompe a metà? A vale 90, B vale ancora 50: la somma dei saldi non è più costante.

TX2 CICLO 3 · TRANSAZIONI

Transazioni multi-documento: requisiti, isolamento, limiti

- Richiedono un **replica set**. Si aprono in una **sessione**, e **ogni operazione** della transazione deve passare la sessione. Si chiudono con `commitTransaction()` o `abortTransaction()`.
- Isolamento: **snapshot**. Le modifiche diventano visibili agli altri **solo al commit**, tutte insieme.
- Limiti (default): `transactionLifetimeLimitSeconds` = **60 s**; `maxTransactionLockRequestTimeoutMillis` = **5 ms** (attesa massima di un lock).
- Costi: le versioni dei documenti restano in cache per tutta la durata; i lock sui documenti modificati sono tenuti fino al commit.

MONGOSH

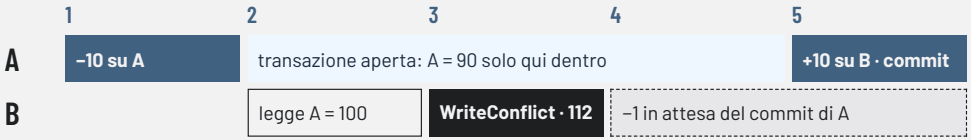
```
const s = db.getMongo().startSession()
const acc = s.getDatabase('shop').lab_accounts
s.startTransaction()
acc.updateOne({ _id: 'A' }, { $inc: { balance:
NumberDecimal('-10.00') } })
acc.updateOne({ _id: 'B' }, { $inc: { balance:
NumberDecimal('10.00') } })
s.commitTransaction()
```

TX3 CICLO 3 · TRANSAZIONI

Isolamento e `WriteConflict`

- Le modifiche di una transazione restano **invisibili** agli altri fino al commit (snapshot isolation).
- Se una seconda transazione tocca lo **stesso documento**, MongoDB non la fa attendere: fallisce subito con `WriteConflict` ed etichetta `TransientTransactionError`.

LINEA DEL TEMPO · DUE CORSIE



MOMENTO	SESSIONE	OPERAZIONE	EFFETTO
1	A	in transazione, toglie 10 al conto A	A vale **90** solo dentro la transazione
2	B	legge il conto A, fuori da transazioni	vede ancora **100**
3	B	in transazione, scrive sul conto A	`WriteConflict` immediato
4	B	fuori da transazioni, scrive sul conto A	**resta in attesa**
5	A	commit	B si sblocca

TX4 CICLO 3 · TRANSAZIONI

`withTransaction` in Java; PostgreSQL vs MongoDB

JAVA

```java
try (ClientSession session = client.startSession()) {
    session.withTransaction(() -> {
        accounts.updateOne(session, eq("_id", "A"), inc("balance", new Decimal128(new BigDecimal("-10.00"))));
        accounts.updateOne(session, eq("_id", "B"), inc("balance", new Decimal128(new BigDecimal("10.00"))));
        return null;
    }, TransactionOptions.builder().writeConcern(WriteConcern.MAJORITY).readConcern(ReadConcern.SNAPSHOT).build());
}
```

+ `withTransaction` **ritenta** da solo su `TransientTransactionError` (e il commit su `UnknownTransactionCommitResult`), entro 120 s. Il corpo deve essere **rieseguibile**: niente email o chiamate HTTP dentro.

DUE TRANSAZIONI SULLA STESSA RIGA/DOCUMENTO	POSTGRESQL (GIORNO 1)	MONGODB
Comportamento	la seconda **attende** il lock, poi procede o fallisce con `40001`	la seconda **fallisce subito** (`WriteConflict`)
Chi ritenta	l'applicazione	il driver, con `withTransaction`

02

CICLO 3 · LABORATORIO

Modellazione

Quattro esercizi su ShopIT: inserimento dei prodotti, pattern computed e subset, extended reference. In chiusura, correzione in plenaria.

L3 CICLO 3 · CONSEGNA

Consegna Laboratorio 3

+ **Fase 3: modellazione** sul database `shop` di ShopIT: i prodotti partono con 60 recensioni annidate.

TEMA	#	ESERCIZIO
Prodotti	1	Inserisci con `insertMany` tre nuovi prodotti (`P-101`…`P-103`) con `name`, `price`, `category` e un array `reviews` di 2–3 recensioni annidate (`stars`, `customerId`, `createdAt`).
Computed	2	Calcola `rating: { count, sum }` per ogni prodotto; il voto medio si ricava in lettura.
Subset	3	Aggiungi a ogni prodotto `lastReviews`: le ultime 5 recensioni, la più recente in testa.
Extended ref.	4	Rinomina il cliente 4242 e riallinea `customer.name` in tutti i suoi ordini con un `updateMany` rieseguibile.

03

CICLO 4 · TEORIA

Indici, `explain()`, aggregation e view, confronto con PostgreSQL

IX1 CICLO 4 · INDICI

Indici: `COLLSCAN`, `IXSCAN` e come leggere `explain`

- Senza un indice utile il server legge **ogni documento**: stage `COLLSCAN` (il `Seq Scan` del Giorno 1). Un indice è un **B-Tree** delle chiavi, con il riferimento al documento; serve filtri **e** ordinamenti. `_id` ha sempre il suo indice univoco.
- `explain()` ha tre livelli: `queryPlanner` (non esegue), **`executionStats`** (esegue, con i contatori reali), `allPlansExecution` (anche i piani scartati).

Regola dei tre numeri: il caso ideale è `nReturned` ≈ `totalKeysExamined` ≈ `totalDocsExamined`. Se i documenti esaminati superano di molto quelli restituiti, il server legge e scarta.

MONGOSH

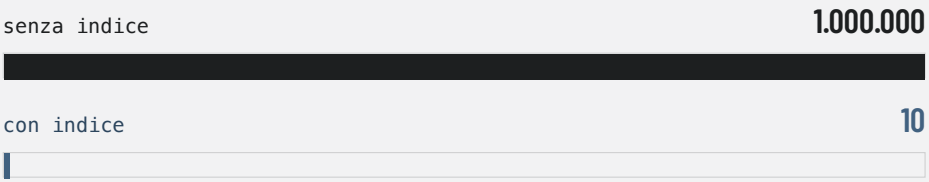
```
db.orders.find({ customerId: 4242
}).explain('executionStats').executionStats
```

OUTPUT

```
// senza indice                    // con { customerId: 1 }
stage: 'COLLSCAN'                  stage: 'FETCH' ← 'IXSCAN'
nReturned:              10         nReturned:              10
totalKeysExamined:       0         totalKeysExamined:      10
totalDocsExamined: 1000000         totalDocsExamined:      10
executionTimeMillis:   412         executionTimeMillis:     0
```

CAMPO DI `executionStats`	SIGNIFICATO
`winningPlan` e stage	albero dal basso verso l'alto: `COLLSCAN`, `IXSCAN`, `FETCH`, `SORT`, `LIMIT`, `PROJECTION_COVERED`
`nReturned`	documenti restituiti
`totalKeysExamined`	voci dell'indice lette
`totalDocsExamined`	documenti letti

TOTALDOCSEXAMINED

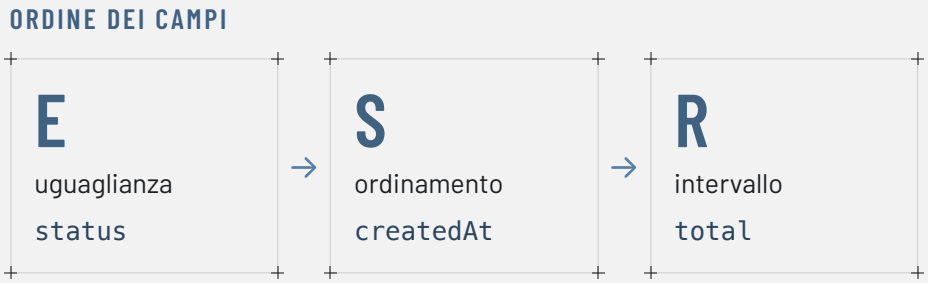


nReturned = 10 in entrambi i casi

IX3 CICLO 4 · INDICI

Indici composti e regola ESR

- Indice composto: chiavi ordinate per il **primo** campo, poi per il secondo a parità del primo (la regola dell'elenco telefonico). Serve anche le query sui **prefissi** (`{ a, b }` serve `a`, non `b`).
- **ESR**, l'ordine dei campi: **E**quality (uguaglianze), **S**ort (ordinamento), **R**ange (intervalli).



MONGOSH

```
// status = 'pending', total >= 500, ordinati per createdAt (desc), primi 10
```

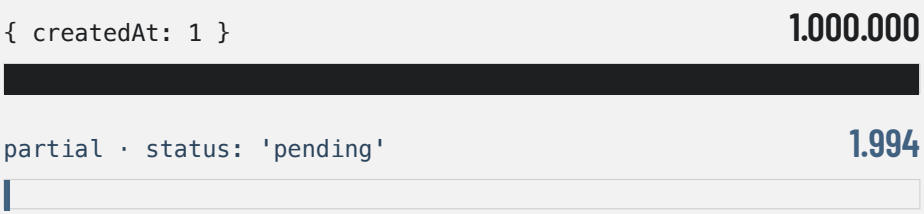
INDICE	ORDINE	STAGE	totalKeysExamined	totalDocsExamined
`{ status, total, createdAt }`	E-R-S	`FETCH` ← **`SORT`** ← `IXSCAN`	1.046	10
`{ status, createdAt, total }`	**E-S-R**	`LIMIT` ← `FETCH` ← `IXSCAN`	17	10

Multikey, unique, partial, TTL

TIPO	COSA FA	DA RICORDARE
Multikey (`{ tags: 1 }`)	una voce **per elemento** dell'array	il parallelo dell'indice GIN. `find({ tags: 'vip' })`: 1.000 chiavi, 1.000 risultati. **Al massimo un campo array** in un indice composto: `{ tags: 1, 'items.sku': 1 }` → `cannot index parallel arrays`
Unique	rifiuta i duplicati (11000)	la creazione **fallisce** se ci sono già duplicati; un documento **senza** il campo conta come `null`. È l'unico modo di garantire l'univocità
Partial	indicizza solo i documenti che soddisfano un filtro	`{ createdAt: 1 }` con `status: 'pending'`: 1.994 voci invece di 1 milione. Usato **solo** se la query contiene una condizione compatibile col filtro
TTL	elimina i documenti più vecchi della soglia	solo campi di tipo `Date`; una data come stringa **non scade mai**; il processo gira **ogni 60 s**

QUERY	INDICE USATO
`find({ status: 'pending' }).sort({ createdAt: -1 }).limit(5)`	partial: **5** chiavi
`find({ status: 'paid' }).sort({ createdAt: -1 }).limit(5)`	nessuno: `COLLSCAN` + `SORT` su **1.000.000** documenti

VOCI DELL'INDICE



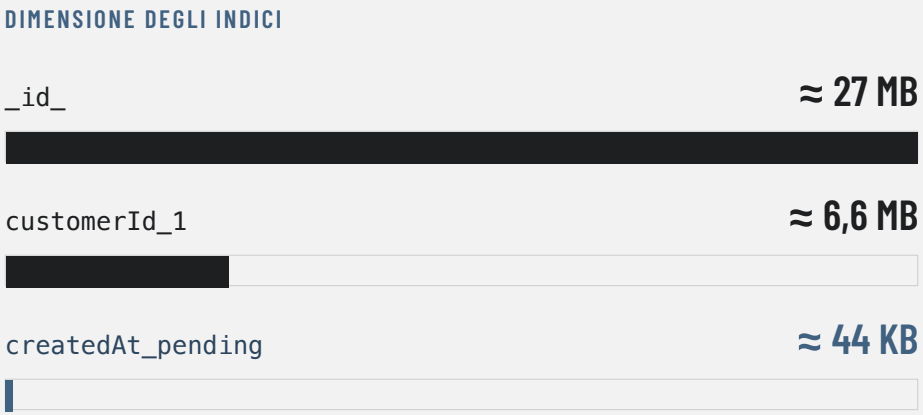
IX5 CICLO 4 · INDICI

Covered query, plan cache, costo degli indici

+ **Covered query**: filtro **e** proiezione usano solo campi dell'indice → il server risponde **senza leggere i documenti** (`PROJECTION_COVERED`, `totalDocsExamined: 0`). Va escluso `_id` dalla proiezione, se non è nell'indice. È l'`Index Only Scan`.

PROIEZIONE (INDICE SU customerId)	STAGE	totalDocsExamined
`{ _id: 0, customerId: 1 }`	PROJECTION_COVERED ← IXSCAN	**0**
`{ _id: 0, customerId: 1, total: 1 }`	PROJECTION_SIMPLE ← FETCH ← IXSCAN	10

+ **Plan cache**: con più indici candidati, i piani **competono** e il vincitore va in cache, associato alla **forma** della query. La voce nasce **inattiva** e si attiva alla seconda esecuzione. Si svuota alla creazione o eliminazione di un indice, o con `getPlanCache().clear()`.
+ **Costo**: ogni indice si aggiorna a **ogni scrittura** e occupa cache. Si cercano gli inutilizzati con `$indexStats`, e si nascondono con `hideIndex` prima di eliminarli.



Pipeline e stage: `find` vs `aggregate` e l'ordine logico SQL

MONGOSH

```
db.orders.aggregate([
  { $match: { status: 'delivered' } },                  // filtra
  { $group: { _id: '$customerId', n: { $sum: 1 } } },   // raggruppa
  { $sort: { n: -1 } },                                 // ordina
  { $limit: 5 }                                         // limita
])
```

- Un array di **stage**: l'output di ogni stage è l'input del successivo. La forma dei documenti cambia lungo la pipeline.
- Le espressioni: `'$total'` è il **valore** del campo; `'$items.qty'` su un array è l'array delle quantità; operatori `$multiply`, `$size`, `$sum`; variabili `'$$this'`, `'$$ROOT'`.

SQL (ordine logico, giorno 1)	Pipeline
`FROM` / `JOIN`	collezione + `$lookup` / `$unwind`
`WHERE`	`$match` (prima di `$group`)
`GROUP BY` + aggregati	`$group`
`HAVING`	`$match` (dopo `$group`)
`SELECT`	`$project` / `$set`
`ORDER BY`, `LIMIT`	`$sort`, `$limit`

DOCUMENTI TRA UNO STAGE E L'ALTRO



AG2 CICLO 4 · AGGREGATION

$group, $project/$set, HAVING

- **$group**: _id è la chiave (un campo, un'espressione o un **documento** per più chiavi; null = un solo gruppo); gli altri campi sono **accumulatori** ($sum, $avg, $min, $max, $count, $push, $addToSet). L'output contiene **solo** _id e gli accumulatori. È uno stage **bloccante**.
- **$project** sceglie cosa tenere (e rinomina, calcola); **$set** ($addFields) aggiunge senza togliere.
- **HAVING = $match dopo $group**: filtra i gruppi.
- Date in **UTC**: $dateTrunc, $year, $month accettano timezone. Somme di Decimal128 **esatte**.

MONGOSH

```
db.orders.aggregate([ { $group: { _id: '$status', n: { $sum: 1 } } }, { $sort: { n: -1 } } ])
// [ { _id: 'delivered', n: 899915 }, { _id: 'shipped', n: 59772 }, { _id: 'paid', n: 30272 },
//   { _id: 'cancelled', n: 8047 }, { _id: 'pending', n: 1994 } ]

db.orders.aggregate([ { $group: { _id: '$customerId', ordini: { $sum: 1 } } }, { $match: { ordini: { $gte: 25 } } }, { $sort: { _id: 1 } } ])
// [ { _id: 61325, ordini: 25 }, { _id: 61731, ordini: 25 }, { _id: 80617, ordini: 25 } ]
```

AG3 CICLO 4 · AGGREGATION

$unwind e $lookup

- **$unwind**: un documento **per ogni elemento** dell'array (gli altri campi si ripetono). Con **preserveNullAndEmptyArrays: true** i documenti con array vuoto o assente restano: è la LEFT JOIN (senza, è una INNER JOIN).
- **$lookup**: from, localField, foreignField, as. Il risultato è **sempre un array** (vuoto se non c'è corrispondenza: è una LEFT JOIN); di solito segue $unwind. Con pipeline si filtra, ordina e limita i documenti uniti.
- **Prestazioni**: serve un **indice** su foreignField, e si filtra **prima** di unire.

MONGOSH

```
db.orders.aggregate([ { $unwind: '$items' },
  { $group: { _id: '$items.sku', pezzi: { $sum: '$items.qty' }, ricavi: { $sum: { $multiply: ['$items.qty',
'$items.unitPrice'] } } } },
  { $sort: { ricavi: -1 } }, { $limit: 3 } ])
// [ { _id: 'SKU-15', pezzi: 30838, ricavi: Decimal128('1585612.43') }, { _id: 'SKU-13', … }, { _id: 'SKU-25', … } ]

db.orders.aggregate([ { $match: { _id: { $lte: 3 } } },
  { $lookup: { from: 'customers', localField: 'customerId', foreignField: '_id', as: 'cliente' } },
  { $unwind: '$cliente' }, { $project: { email: '$cliente.email', country: '$cliente.country' } } ])
// [ { _id: 1, email: 'user44830@example.com', country: 'IT' }, { _id: 2, … }, { _id: 3, … } ]
```

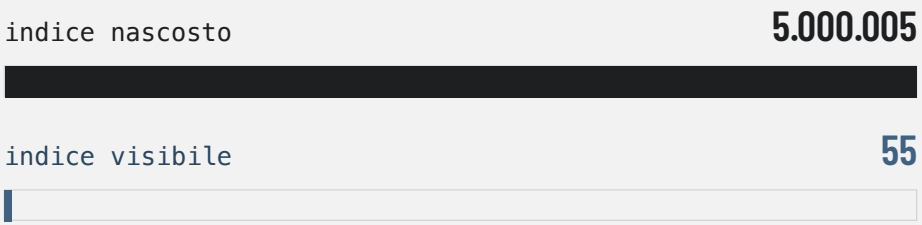
Demo: `hideIndex` e `$lookup`

MONGOSH

```
const lk = () =>
db.customers.explain('executionStats').aggregate([
  { $match: { _id: { $lte: 5 } } },
  { $lookup: { from: 'orders', localField: '_id',
foreignField: 'customerId', as: 'ordini' } },
  { $project: { n: { $size: '$ordini' } } } ])
db.orders.hideIndex('customerId_1')      // l'indice
esiste e si mantiene, ma il planner lo ignora
lk()                                      // stage
$lookup
db.orders.unhideIndex('customerId_1')
lk()
```

CAMPO (STAGE $lookup)	INDICE NASCOSTO	INDICE VISIBILE
strategy	NestedLoopJoin	IndexedLoopJoin
collectionScans	5	0
totalDocsExamined	**5.000.005**	**55**
tempo	‹secondi›	‹millisecondi›

TOTALDOCSEXAMINED · 5 CLIENTI

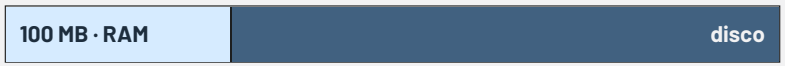


Risultato della pipeline (uguale nei due casi): `[ { _id: 1, n: 14 }, { _id: 2, n: 9 }, { _id: 3, n: 6 }, { _id: 4, n: 13 }, { _id: 5, n: 8 } ]`.

Ottimizzazione: `$match` in testa, memoria, `maxTimeMS`

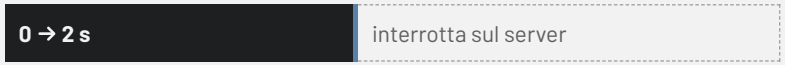
- Solo i **primi** stage possono usare gli indici (`$match`, `$sort`). Dopo `$group`, `$unwind` o un `$project` che calcola, i documenti non esistono più nella collezione.
- Il server **sposta da solo** un `$match` prima di `$project` o `$group` se riguarda campi non modificati (per esempio la chiave di raggruppamento); **non può** farlo su un campo **calcolato**. Non conviene contarci: il `$match` si scrive **in testa**.
- `explain` di una pipeline (8.0): se è eseguita tutta dal motore di query ha la forma di un `find`; altrimenti un array `stages`. Si cerca sempre: indice usato e documenti esaminati.
- **Memoria**: gli stage bloccanti (`$sort`, `$group`) hanno un limite di **100 MB**; oltre, **disco** (attivo di default dalla 6.0). Con `allowDiskUse: false` la pipeline **fallisce** (`QueryExceededMemoryLimitNoDiskUseAllowed`). `$sort` seguito da `$limit` è un **top-k** e resta in pochi KB.
- **`maxTimeMS`**: tempo massimo **lato server**; oltre, `MaxTimeMSExpired` (codice 50).

$SORT SU 1M ORDINI · ≈ 340 MB



☐ `$sort` + `$limit`: top-k, pochi KB

MAXTIMEMS: 2000



`MaxTimeMSExpired · codice 50`

AG6 CICLO 4 · AGGREGATION

View: una pipeline con un nome

- `db.createView(nome, sorgente, pipeline)` salva una pipeline con un nome: si interroga come una collezione (`find`, `aggregate`, Compass).
- **Sola lettura** e **nessun dato salvato**: a ogni lettura la pipeline viene rieseguita sulla collezione sorgente. È il `CREATE VIEW` del Giorno 1.
- Il filtro del `find` si **accoda** alla pipeline della view: gli indici usabili sono quelli della **sorgente**, solo se i primi stage lo permettono (AG5). Una view **non si indicizza**.

MONGOSH

```
db.createView('orders_with_customer', 'orders', [
  { $lookup: { from: 'customers', localField: 'customerId', foreignField: '_id', as: 'c' } },
  { $set: { customerName: { $first: '$c.name' } } },
  { $project: { c: 0 } }
])
db.orders_with_customer.find({ customerId: 4242 })
```

AG7 CICLO 4 · AGGREGATION

View o `$merge`: ricalcolare o materializzare

+ **`$merge`** è l'ultimo stage di una pipeline: scrive il risultato in una collezione, aggiornando i documenti esistenti e inserendo i nuovi. È la `MATERIALIZED VIEW` del Giorno 1, ma **incrementale**: si può ricalcolare solo l'ultimo periodo.

	VIEW (`createView`)	COLLEZIONE CON `$merge`
Dati	sempre aggiornati	aggiornati all'ultima esecuzione
Costo in lettura	l'intera pipeline, ogni volta	una lettura normale
Costo in scrittura	nessuno	si riesegue la pipeline (a mano o pianificata)
Indici	quelli della sorgente	propri, si creano come su ogni collezione
Quando	letture rare o dati che devono essere freschi	report pesanti letti spesso

MONGOSH

```
db.orders.aggregate([
  { $match: { status: 'delivered' } },
  { $group: { _id: '$customer.country', revenue: { $sum: '$total' } } },
  { $merge: { into: 'revenue_by_country', whenMatched: 'replace', whenNotMatched: 'insert' } }
])
```

CF1 CICLO 4 · CONFRONTO

Confronto PostgreSQL vs MongoDB

	POSTGRESQL	MONGODB
Unità e schema	riga in tabella, schema obbligatorio	documento in collezione, schema facoltativo (`$jsonSchema`)
Progetto	normalizzazione, indipendente dalle query	aggregati, guidato dalle query
Relazioni	`FOREIGN KEY`, `JOIN`	embedding, riferimenti, `$lookup`
Atomicità di base	l'istruzione (anche su milioni di righe)	il singolo documento
Transazioni	sempre; conflitto: attesa del lock, poi `40001`	multi-documento su replica set, limite 60 s; conflitto: `WriteConflict` immediato
Planner e indici	a **costi**; B-Tree, GIN, GiST, BRIN	**multi-planning**; B-Tree (multikey, composti, partial, TTL…)
Scalabilità	un primary, **verticale**; repliche in lettura	replica set + **sharding** nativo
Failover	strumenti esterni (Patroni)	nativo, elezione in pochi secondi
Versioni vecchie dei dati	nel file della tabella: `VACUUM`	in cache e history store: **nessun** `VACUUM`
Backup	`pg_dump`, `pg_basebackup` + WAL	`mongodump`, snapshot; point-in-time con strumenti Enterprise o Atlas

CF2 CICLO 4 · CONFRONTO

Quando usare cosa

SCEGLI POSTGRESQL QUANDO…	SCEGLI MONGODB QUANDO…
le relazioni e i vincoli tra entità sono il cuore del dominio	i dati sono naturalmente aggregati e letti insieme
servono query ad hoc, reporting, `JOIN` arbitrari	i pattern di accesso sono noti e stabili
le transazioni toccano spesso più entità	lo schema varia o evolve spesso
il volume sta comodamente su un nodo	servono distribuzione e alta disponibilità native

Ordini, pagamenti, magazzino

invarianti tra entità, reporting

→ **PostgreSQL**

Catalogo prodotti

attributi diversi per categoria, letture molto più frequenti delle scritture

→ **MongoDB**

Telemetria

time series collections e TTL, oppure partizionamento per data e BRIN

→ **dipende**

Nel dubbio: PostgreSQL con **`jsonb`** per la parte flessibile; MongoDB quando la parte documentale diventa il modello principale.

04

CICLO 4 · LABORATORIO

Indici e aggregation

Sei esercizi su ShopIT: indici, `explain`, aggregation e view. In chiusura, correzione in plenaria.

L4 CICLO 4 · CONSEGNA

Consegna Laboratorio 4

+ **Fase 4: indici, aggregation e view** sul database `shop` di ShopIT: si parte senza indici secondari.

TEMA	#	ESERCIZIO
Explain	1	Misura con `explain('executionStats')` gli ordini del cliente 4242 e gli ordini `delivered` di un paese per data. Annota `nReturned`, chiavi e documenti esaminati.
Indici	2	Crea un indice composto per la seconda query seguendo la regola ESR; rimisura.
Indici	3	Rendi **covered** la query del cliente 4242 con indice e proiezione: `totalDocsExamined` deve essere 0.
Aggregation	4	Fatturato per paese degli ordini `delivered`, solo i paesi oltre 10.000 € (`$group` + `$match`).
`$lookup`	5	Per i primi 10 clienti, numero di ordini e totale speso con `$lookup`; verifica in `explain` la strategia usata.
View	6	Salva la pipeline dell'esercizio 5 come view e interrogala con un `find` sul cliente 4242: con `explain`, quale indice usa?

05

CHIUSURA

Q&A e chiusura

Q1 Q&A SULLE DUE GIORNATE

Domande?

GIORNO 1

PostgreSQL

GIORNO 2

MongoDB

Il confronto

Q2 CHIUSURA

Chiusura

- Materiali: slide, soluzioni degli esercizi, appendice di riferimento.
- L'ambiente resta utilizzabile anche dopo il corso.
- Grazie.